

Pad Ring Generation with LibreLane for the IHP SG13G2 Process

Method, driver script (line by line), and the generated pad-ring layout of the chip_top demonstrator

Koen Van Caekenberghe, Ph.D. — ChipDesign B.V.
info@chipdesign.be

2026-07-22

Contents

1. Scope and method	1
2. The IHP SG13G2 I/O cell library	2
3. How the pad ring is placed	3
4. The generation script, line by line	3
5. Defining the ring in Verilog (line by line)	5
6. Assigning pads to sides in config.yaml	6
7. Generated pad-ring layout	7
8. Verification and next steps	10
Appendix A — reproduction	10
Appendix B — Third-party tools and libraries	11

1. Scope and method

This report documents how a **pad ring** is generated for a chip on the **IHP SG13G2** 130 nm open-source process using **LibreLane** — the successor to OpenLane 2. A pad ring is the outer belt of I/O cells that surrounds the core: it carries the bond pads, the ESD structures, the level shifters between the 1.2 V core and the 3.3 V outside world, and the power/ground rails that ring the die. Without it, a hardened core has nothing to be wired or probed through.

The approach here is **fully integrated**: rather than assembling the ring in a separate GDS tool, we describe the ring *structurally in Verilog* (one I/O-cell instance per chip pin), hand that top level to LibreLane’s **Chip flow**, and let its **OpenROAD.PadRing** step place the pads, corners and fillers and connect the ring rails by abutment. The

same netlist that defines the ring is therefore the one that is later placed, routed and verified — there is no manual GDS step to keep in sync.

The worked example is `chip_top`, a 32-pad ring around a small placeholder core (`demo_core`). Everything below is reproducible from the files in this repository with a single command (§4).

Item	Value
Process	IHP SG13G2 (130 nm), PDK=ihp-sg13g2
Tool	LibreLane 3.1, flow Chip, step OpenROAD.PadRing
I/O library	sg13g2_io (libs.ref/sg13g2_io)
Pad cell footprint	80 μm (along ring) $\times$ 180 μm (deep)
Corner cell	180 $\times$ 180 μm
Pad-to-edge (seal-ring) spacing	PAD_EDGE_SPACING = 140 μm
Pad pitch	150 μm , uniform on all four sides
Die	1910 $\times$ 1910 μm , 8 pads per side
Result	32 I/O pads + 4 corners + 72 fillers, ring closed by abutment

2. The IHP SG13G2 I/O cell library

The pad ring is built exclusively from cells in `libs.ref/sg13g2_io`. LibreLane learns about them automatically: the PDK ships a LibreLane configuration (`libs.tech/librelane/sg13g2_io/config.tcl`) that registers the pad LEF/GDS/Verilog, the placement **sites** (`sg13g2_ioSite`, `sg13g2_cornerSite`), the corner and filler cell names, and the 140 μm edge spacing. The relevant cell classes are:

Class	Cells	Purpose
Input	sg13g2_IOPadIn	pad $\rightarrow$ core receiver
Output	sg13g2_- IOPadOut{4,16,30}mA	core $\rightarrow$ pad driver, three drive strengths
Tri-state	sg13g2_- IOPadTriOut{4,16,30}mA	driver with output-enable
Bidirectional	sg13g2_- IOPadInOut{4,16,30}mA	driver + receiver + enable
Analog	sg13g2_IOPadAnalog	direct pad $\leftrightarrow$ core analog connection
Power / ground	sg13g2_IOPad{Vdd, Vss, IOVdd, IOVss}	core (1.2 V) and I/O (3.3 V) supply pads
Corner	sg13g2_Corner	die-corner cell that turns the ring 90°
Filler	sg13g2_Filler{200...10000}	close the gaps between pads and continue the rails

A note on **bond pads**: the IHP I/O cells already carry their wire-bond opening *inside* the pad cell (the Liberty attribute `bond_pads : 1` is set on every `sg13g2_-IOPad*`). There is no separate bond-pad cell in the PDK. The bond wire therefore lands directly on the pad cell, and the flow is told not to look for a stand-alone bond-pad master (§6, `PAD_BONDPAD_NAME: null`).

3. How the pad ring is placed

The `OpenROAD.PadRing` step runs the PDK-agnostic algorithm in `libre-lane/scripts/openroad/common/pad_cfg.tcl`. Conceptually, for each of the four sides it:

1. builds an I/O placement row inset from the die edge by `PAD_EDGE_SPACING` (140 μm), leaving room for the seal ring;
2. sums the widths of the pads assigned to that side and checks they fit;
3. distributes the free space *evenly* — the inter-pad gap is rounded down to the 1 μm minimum filler granularity, and the two end gaps to the corners absorb the remainder;
4. places each pad instance at the computed coordinate with the correct rotation for that side;
5. drops the four `sg13g2_Corner` cells and back-fills every remaining gap with the largest fitting `sg13g2_Filler*`;
6. calls `connect_by_abutment`, so the power, ground and ESD rails that run through every cell join into four continuous rings simply because the cells touch.

The engineer therefore controls only two things: **which pad instances exist** (the Verilog, §5) and **which side each one sits on, in what order** (the `PAD_*` lists, §6). The spacing is computed.

Setting the pad pitch. Because the pads are spread evenly, the centre-to-centre pitch is not a direct configuration knob — it *emerges* from the die size. For a target pitch p with pad width w_{pad} and N pads on a side, the inter-pad gap must be $g = p - w_{pad}$, and the die side that produces it exactly is

$$\text{side} = 2 \cdot 140 + 2 \cdot 180 + N w_{pad} + (N+1) g$$

For the required **150 μm pitch** with 80 μm pads and $N = 8$: $g = 70 \mu\text{m}$ and $\text{side} = 280 + 360 + 640 + 9 \cdot 70 = \mathbf{1910 \mu\text{m}}$. Two divisibility conditions make the pitch *exact* rather than approximate: the algorithm floors the gap to the 1 μm site width ($630/9 = 70$ exactly, no loss), and the two end gaps $(630 - 7 \cdot 70)/2 = 70 \mu\text{m}$ land on the site grid. Off-grid choices are silently floored — the pitch then comes out slightly below target — so the die should always be derived from the pitch, not the other way around.

4. The generation script, line by line

The whole flow is driven by `gen_padring.sh`:

```
1  #!/bin/bash
2  # gen_padring.sh – generate the pad ring with LibreLane (IHP SG13G2)
3  set -euo pipefail
4  cd "$(dirname "$0")"
5
6  # --- 1. Environment: PDK location, tool paths ---
```

```

7  . ./env.sh
8
9  # --- 2. Run the Chip flow up to the pad-ring step ---
10 librelane config.yaml \
11     --pdk "$PDK" --pdk-root "$PDK_ROOT" --manual-pdk \
12     --to OpenROAD.PadRing \
13     "$@"
14
15 # --- 3. Report where the result landed ---
16 RUN=$(ls -dt runs/RUN_* | head -1)
17 echo "Pad ring written to: $RUN/16-openroad-padring/chip_top.def"

```

Line 3 — `set -euo pipefail` makes the script fail fast: abort on any command error (-e), on an undefined variable (-u), and on a failure anywhere in a pipeline (-o pipefail). A silent half-finished pad ring is worse than a clean stop.

Line 4 — `cd` to the script’s own directory so the relative paths in `config.yaml` (which use LibreLane’s `dir::` prefix) resolve regardless of where the script is invoked from.

Line 7 — source `env.sh`, the single place where the environment is configured. It exports `PDK_ROOT` and `PDK` (defaulting to a standard installation, both overridable from the caller’s environment) and prepends common EDA tool locations to `PATH` *when they exist* — many setups do not expose `yosys`, `openroad` and `klayout` to non-interactive shells, and without this LibreLane’s sub-processes fail with “*command not found*”. On systems where the tools are already on `PATH` the loop is a no-op, so the script stays portable.

Line 10 — invoke LibreLane on the design’s `config.yaml`. Everything about *what* is built lives in that file (§6); the command line only says *how far* and *against which PDK*.

Line 11 — `--pdk ... --pdk-root ... --manual-pdk`. The critical flag is `--manual-pdk`: it tells LibreLane to use the PDK already on disk instead of trying to download a version-pinned copy through `ciel`. Without it the run can die trying to write into a read-only version-managed PDK cache.

Line 12 — `--to OpenROAD.PadRing` stops the flow immediately after the pad ring is placed. This is the “pad-ring only” mode: synthesis → floorplan → power-net setup → **pad ring**, then halt, so pad placement can be inspected and iterated in seconds without paying for full place-and-route. Dropping this flag runs the complete Chip flow through to a signed-off chip GDS (seal ring, filler, density and antenna checks included).

Line 13 — “`$@`” forwards any extra arguments to LibreLane (e.g. `--to Odb.SetPowerConnections` to stop even earlier, or nothing to run to `PadRing`).

Lines 16-17 — find the newest run directory and print the path of the emitted pad-ring DEF — the artifact rendered in §7.

Run it with:

```
./gen_padring.sh
```

5. Defining the ring in Verilog (line by line)

The pad ring’s *contents* are one I/O-cell instantiation per pin in the chip top (src/chip_top.v). The instance **names** chosen here are exactly the handles the placer uses in §6. A representative slice:

```
1  (* keep *) sg13g2_IOPadIOVdd sg13g2_IOPad_iovdd_w (  
2  `ifdef USE_POWER_PINS  
3      .vss(VSS), .vdd(VDD), .iovss(IOVSS), .iovdd(IOVDD)  
4  `endif  
5  );  
6  
7  sg13g2_IOPadIn sg13g2_IOPad_io_clk (  
8      .p2c(clk_i), .pad(clk_PAD)  
9  );  
10  
11 sg13g2_IOPadOut30mA sg13g2_IOPad_io_done (  
12     .c2p(done_o), .pad(done_PAD)  
13 );  
14  
15 sg13g2_IOPadInOut30mA sg13g2_IOPad_io_gpio (  
16     .c2p(gpio_out), .c2p_en(gpio_oe), .p2c(gpio_in), .pad(gpio_PAD)  
17 );
```

Line 1 — a **power pad**. `(* keep *)` stops synthesis from optimising the cell away: it has no logic function, only supply connections, so without the attribute Yosys would delete it. The instance name `sg13g2_IOPad_iovdd_w` is what appears in `PAD_WEST`.

Lines 2-4 — the four ring supply nets (`vdd`, `vss`, `iovdd`, `iovss`) are wired only under the `USE_POWER_PINS` macro. During logic synthesis the macro is off and the pins float; the physical rails are joined later by `connect_by_abutment`, and the global power connections tie them to the supply nets. This is the standard LibreLane power-pad idiom.

Lines 7-9 — an **input pad**. `sg13g2_IOPadIn` has two signal pins: `pad` is the external bond pad (a top-level port of the chip), and `p2c` (“pad-to-core”) is the received signal handed to the core net `clk_i`.

Lines 11-13 — an **output pad**, drive strength 30 mA. Here the direction reverses: the core drives `c2p` (“core-to-pad”) and the cell drives the external pad.

Lines 15-17 — a **bidirectional pad**. It combines both directions plus an output-enable: `c2p/c2p_en` drive the pad when enabled, and `p2c` always returns the pad value to the core — the three-wire decomposition of an inout.

Buses of pads are built with a generate loop; the resulting instance names take the hierarchical form `sg13g2_IOPad_dout[0].sg13g2_IOPad_io_dout`, which is why those entries are bracket-escaped in the configuration.

The same structure is captured as an **xschem schematic** in `xschem/chip_top.sch`, drawn with the PDK’s own `sg13g2_io` symbols and arranged like the physical ring: the north/south banks along the top and bottom, the west/east banks (including the four supply-pad pairs) at the sides, and the `demo_core` block in the centre. Every pad pin carries its net label, so netlisting the schematic (`cd xschem && xschem --no_x -q -n chip_top.sch`) reproduces the connectivity of `chip_top.v` — all 32 pads with their rail, core-side and bond-pad nets. Corner and filler cells have no schematic view (they are physical-only); the ring rails `VDD/VSS/IOVDD/IOVSS` join by

abutment in layout.

chip_top — IHP SG13G2 pad ring (32 pads @ 150 um pitch, mirrors src/chip_top.v)

Koen Van Caekenberghe (koen.vancaekenberghe@chipdesign.be), ChipDesign B.V., 07 2026 — Apache-2.0
corner and filler cells are physical-only (no schematic view); ring rails VDD/VSS/IOVDD/IOVSS connect by abutment in layout

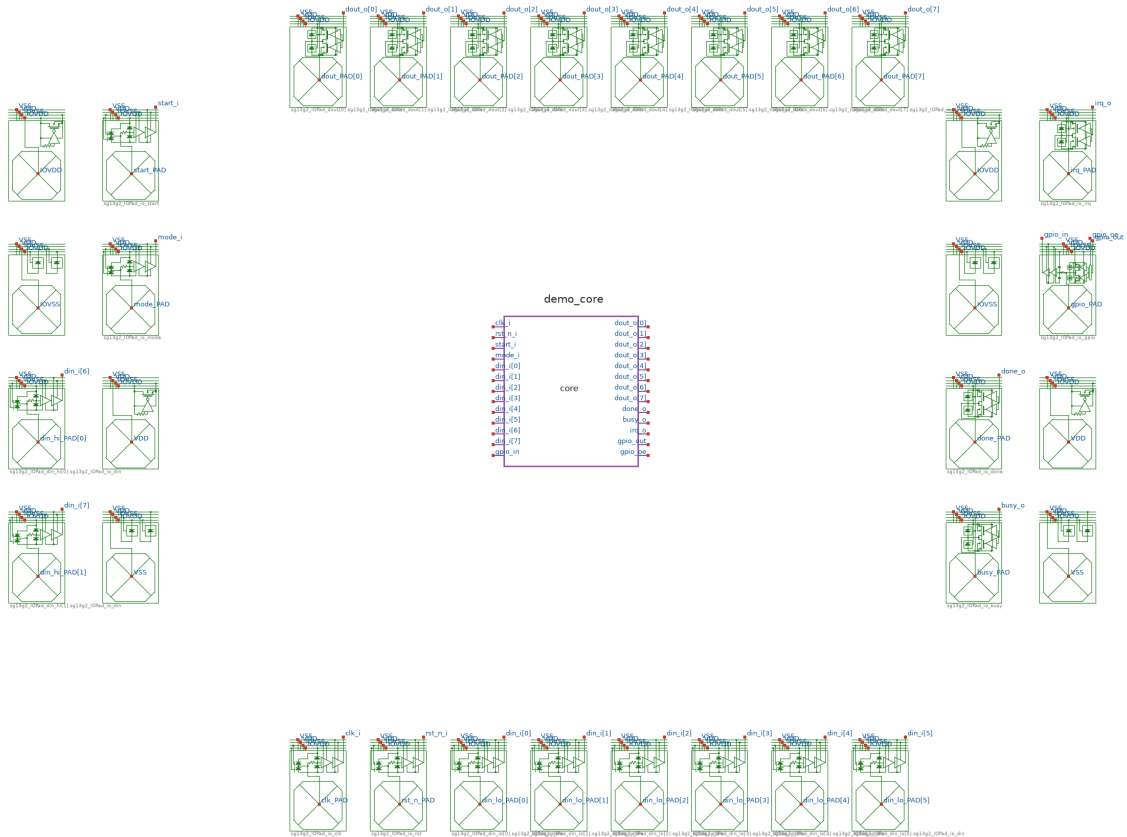


Figure 1: xschem schematic of the pad ring (xschem/chip_top.sch), drawn with the PDK sg13g2_io symbols: north bank (dout pads) on top, south bank (clock, reset, din low bits) at the bottom, west and east banks with the supply pads at the sides, and the demo_core block in the centre. Each pad shows its ESD network, driver/receiver and bond-pad octagon; net labels match src/chip_top.v.

6. Assigning pads to sides in config.yaml

The floorplan-side of the method is the pad configuration. The essential block:

```
1 meta:
2   version: 2
3   flow: Chip
4   substituting_steps:
5     Verilator.Lint: null
6
7   FP_SIZING: absolute
8   DIE_AREA: [0, 0, 1910, 1910]
9
10  PAD_BONDPAD_NAME: null
11
12  PAD_SOUTH: [ sg13g2_IOPad_io_clk, sg13g2_IOPad_io_rst,
```

```

13         "sg13g2_IOPad_din_lo\\[0\\].sg13g2_IOPad_io_din", ... ]
14 PAD_WEST: [ sg13g2_IOPad_iovdd_w, sg13g2_IOPad_iovss_w, ...,
15             sg13g2_IOPad_vdd_w, sg13g2_IOPad_vss_w ]
16 PAD_NORTH: [ "sg13g2_IOPad_dout\\[0\\].sg13g2_IOPad_io_dout", ... ]
17 PAD_EAST: [ sg13g2_IOPad_iovdd_e, sg13g2_IOPad_iovss_e,
18             sg13g2_IOPad_io_done, ..., sg13g2_IOPad_vss_e ]

```

Line 3 — flow: Chip selects LibreLane’s chip-assembly flow. Compared with the standard Classic (macro) flow it inserts OpenROAD.PadRing right after the power-net step and appends the seal-ring, filler and density steps; it also disables ordinary I/O-pin placement, because on a padded chip the *pads* are the boundary terminals.

Line 5 — null out the Verilator.Lint step. It is not needed to build a pad ring and the linter is not reliably available in this container; nulling a step is the LibreLane idiom for “skip this”.

Lines 7-8 — an **absolute** floorplan. FP_SIZING: absolute with DIE_AREA fixes the die at $1910 \times 1910 \mu\text{m}$. Pad-ringd chips are almost always sized absolutely: the die must be large enough for the pads, not for the core. Here the value is *derived from the required $150 \mu\text{m}$ pad pitch* using the formula of §3: $\text{gap} = 150 - 80 = 70 \mu\text{m}$, $\text{side} = 280 + 360 + 8 \cdot 80 + 9 \cdot 70 = 1910 \mu\text{m}$.

Line 10 — PAD_BONDPAD_NAME: null overrides the PDK default (which names a bondpad_70x70 cell that this PDK does not ship) and tells the placer *not* to place a separate bond pad. The IHP pads already contain their bond opening (§2), so this is correct, not a work-around.

Lines 12-18 — the four side lists. Each entry is the **instance name** of a pad from chip_top.v, and the **order** within a list is the placement order along that edge. The design is balanced at 8 pads per side: clocks/reset and the low data bits on the south; the high data bits, control and one supply pair on the west; the output bus on the north; the status outputs, the bidirectional GPIO and the other supply pair on the east. Names produced by a generate loop carry a [i] index and must be escaped \\[i\\] so LibreLane treats them literally.

That is the entire method: **instances** (Verilog) + **placement order** (these lists) + **die size** (DIE_AREA). Everything else — spacing, corners, fillers, rail connection — is computed by pad_cfg.tcl.

7. Generated pad-ring layout

Running ./gen_padring.sh completes the flow through OpenROAD.PadRing and writes the pad-ring DEF. The helper scripts/render_padring.py then re-assembles the placed pad/corner/filler cells into a GDS and rasterises it with the PDK’s KLayout layer colours. (One DEF subtlety the script honours: PLACED (x y) ORIENT anchors the *lower-left corner of the oriented abutment box* at (x, y) — not the cell origin — so the instance transform must compensate for the rotation/mirror of each cell.)

The same placement, coloured by pad *function* rather than by mask layer, makes the floor-planning choices explicit:

The placer’s own report confirms the geometry: each side sums to **640 μm** of pad width ($8 \times 80 \mu\text{m}$), the free space is distributed as an exact **70 μm** gap between

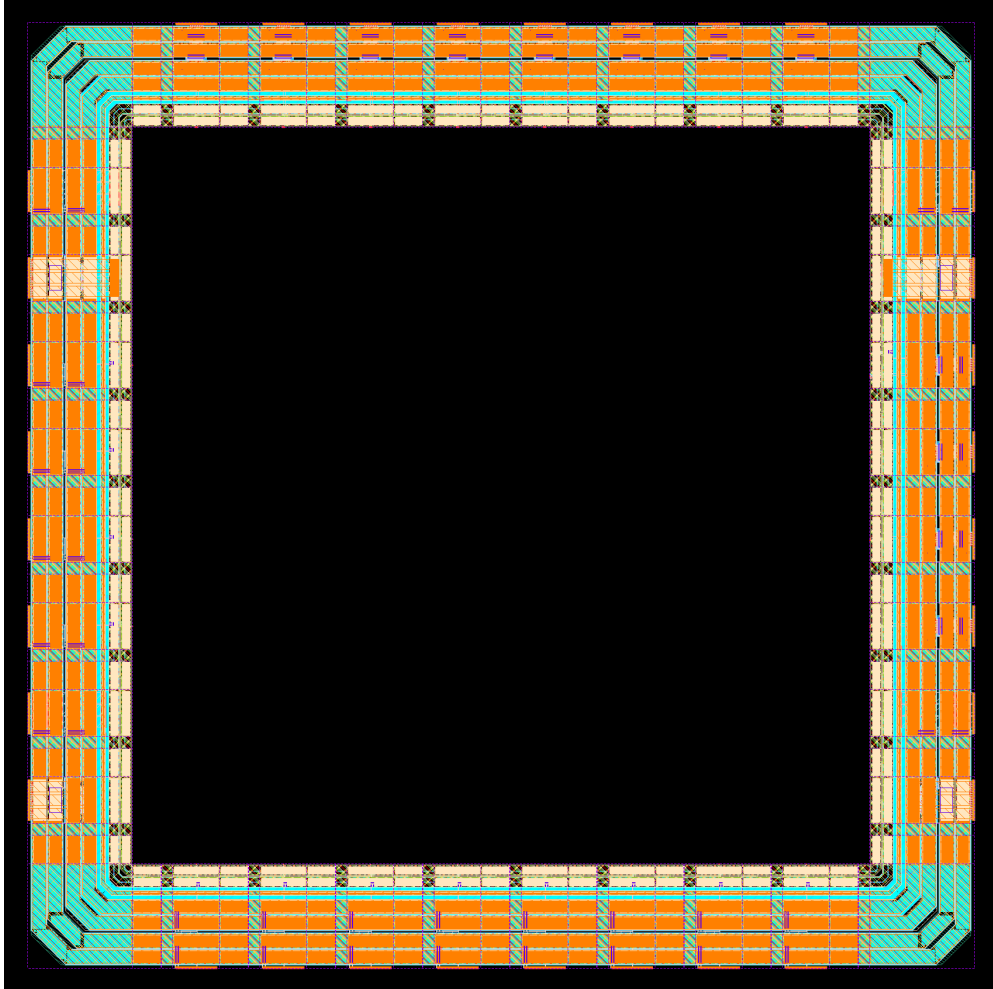


Figure 2: Generated chip_top pad ring at **150 μm pad pitch**, rendered from GDS with the IHP SG13G2 layer palette — a fully *closed* ring: the four chamfered sg13g2_Corner cells turn the ring at the die corners, eight I/O pads occupy each side at 150 μm centre-to-centre spacing, and sg13g2_Filler* cells tile the 70 μm gaps, so the supply and ESD rails run continuously around the die (connect_by_abutment). Geometric check from the DEF: each side is tiled gap-free from corner to corner (140 μm $\rightarrow$ 1770 μm). The core area in the centre is empty because the flow was stopped at the pad-ring step.

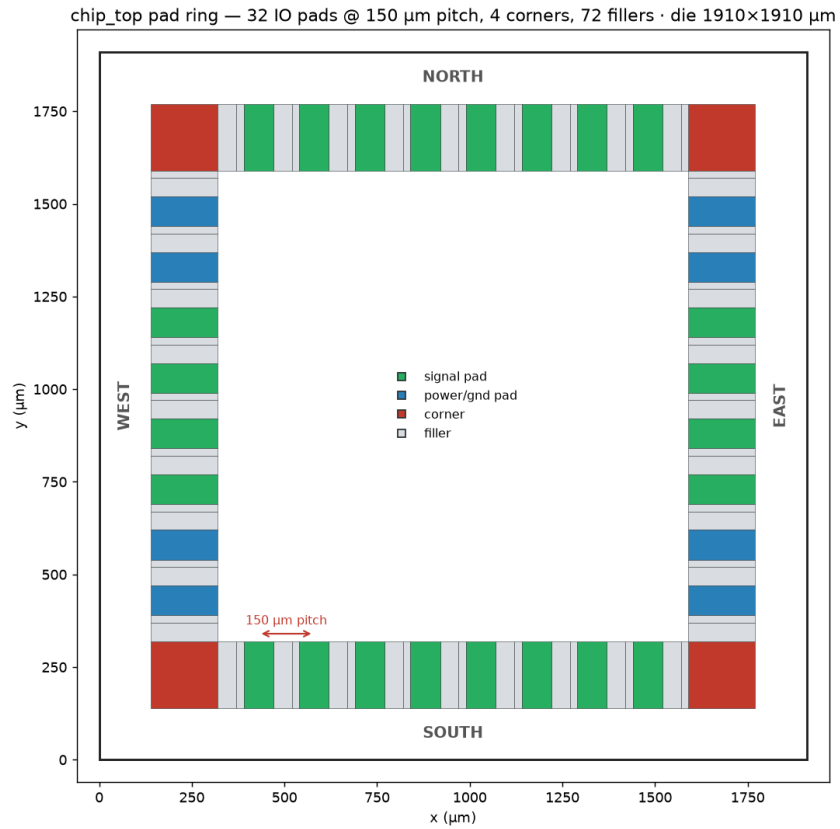


Figure 3: Pad-ring floor plan by function: green = signal pads, blue = power/ground pads, red = corner cells, grey = fillers. 32 I/O pads at a uniform 150 μm pitch + 4 corners + 72 fillers on a 1910 $\times$ 1910 μm die, each pad inset 140 μm from the die edge to leave room for the seal ring.

pads (space_between_pads: 70.0), and the run ends with “Placing corner cells... Placing filler cells... Connecting ring signals...” and **Flow complete**. A centre-to-centre check on the emitted DEF gives a single pitch value of **150.0 µm** on every side (pad centres at 430, 580, ..., 1480 µm). The DEF contains 32 I/O pads (12 inputs, 11 outputs, 1 bidirectional, 8 supply), 4 corners and 72 fillers on the 1910 × 1910 µm die.

8. Verification and next steps

How to check the ring. The fastest check is the pad_cfg.tcl console output: a “sum of cell widths ... larger than the width of this side” error means DIE_AREA is too small, and a “No instance ... found” error means a PAD_* name does not match a Verilog instance. Visually, open the run in the OpenROAD GUI — librelane config.yaml --pdk "\$PDK" --pdk-root "\$PDK_ROOT" --manual-pdk --last-run -f openinopenroad — and confirm four populated sides, four corners, gap-free fillers, and the 140 µm inset.

From pad ring to chip. Removing --to OpenROAD.PadRing runs the full Chip flow: it places and routes the core inside the ring, generates the seal ring, inserts fillers, and runs the density and antenna checks, ending in a final chip GDS under runs/.../final/gds/.

For your own core. Replace the placeholder demo_core with the actual core RTL (or drop in a hardened core macro via MACROS/EXTRA_LEFS), then edit the pad instantiations and the four PAD_* lists to match its true pin-out and drive-strength requirements — the method itself does not change.

Appendix A — reproduction

```
Design      : this repository (pad-ring-ihp)
Files       : src/chip_top.v      chip top, instantiates the pad ring
              src/demo_core.v    placeholder core
              config.yaml        flow = Chip, PAD_* side lists, DIE_AREA
              constraint.sdc      clock defined at the clock pad's p2c pin
              env.sh             PDK_ROOT / PDK, tool-path setup
              gen_padring.sh      driver (see §4)
              scripts/render_padring.py DEF+GDS -> layout PNG (fig below)
              xschem/chip_top.sch xschem schematic of the ring (PDK symbols)
Run         : ./gen_padring.sh
Environment: IIC-OSIC-TOOLS 2026.06 container (all tools + PDK pre-installed);
              any setup with librelane/yosys/openroad/klayout on PATH also works
PDK         : ihp-sg13g2 @ $PDK_ROOT (LibreLane 3.1, --manual-pdk)
Output      : runs/RUN_*/16-openroad-padring/chip_top.def
Check       : scripts/check_padring.py <def> --pitch 150 (PASS required)
Figure      : doc/fig/padring_layout.png (GDS render, PDK colours)
              doc/fig/padring_floorplan.png (function-coloured floor plan)
```

Appendix B — Third-party tools and libraries

All process-specific material in this project — I/O cells, layer stack, KLayout layer palette, LibreLane PDK configuration — comes exclusively from the **IHP SG13G2 open PDK**. No other foundry’s libraries, rules or documentation are used or referenced.

Used to generate and verify the pad ring (all invoked by `gen_padring.sh / scripts/`):

Tool / library	Version	License	Role
IHP-Open-PDK (ihp-sg13g2)	commit 144f811c	Apache-2.0	Process kit: sg13g2_io pad cells (LEF, GDS, Verilog, Liberty), sg13g2_stdcell, LibreLane tech config, KLayout .lyp
— sg13g2_io cell library	(in PDK)	Apache-2.0	I/O pads, corner and filler cells; generated from Chips4Makers c4m-pdk-ihpsg13g2 v0.0.4 plus contributed Liberty/Verilog views
LibreLane	3.1.0.dev1	Apache-2.0	Flow engine: Chip flow, OpenROAD.PadRing step, pad_cfg.tcl placement algorithm
OpenROAD	26Q2-2270- g4c26918f5	BSD-3-Clause	Floorplan, ODB, built-in pad placer (make_io_sites, place_pad, place_corners, place_io_fill, connect_by_- abutment)
Yosys	0.66	ISC	Synthesis of the chip top (pad instances kept via (* keep *))
KLayout (Python module)	0.30.9	GPL-3.0-or-later	GDS assembly and headless rendering (scripts/render_- padring.py)
xschem	3.4.8RC	GPL-2.0-or-later	Schematic capture of the pad ring (xschem/chip_- top.sch, PDK sg13g2_io symbols) and netlist cross-check

Tool / library	Version	License	Role
Python	3.12.3	PSF-2.0	Scripts (<code>check_padring.py</code> , <code>render_padring.py</code> , filters)
IIC-OSIC-TOOLS	2026.06	Apache-2.0	Container image used for development and verification — ships all of the above and the PDK pre-installed (optional: any environment with the tools on PATH works)

Used to build this documentation (`doc/build.sh`):

Tool / library	Version	License	Role
Pandoc	3.1.3	GPL-2.0-or-later	Markdown → LaTeX conversion (default LaTeX template)
Tectonic	0.15.0	MIT	XeTeX-based LaTeX engine, LaTeX → PDF (tectonic, xelatex, lualatex or pdflatex all work)
Matplotlib	3.11.0	Matplotlib License (BSD-style)	Floor-plan figure generation

Notes:

- The full Chip flow (beyond the `OpenROAD.PadRing` stop) additionally uses **Magic**, **Netgen** and the **KLayout DRC/LVS decks** from the IHP PDK — all open-source (and all shipped in IIC-OSIC-TOOLS 2026.06), none foundry-encumbered beyond the IHP PDK itself.
- The `Verilator.Lint` step is disabled in `config.yaml` and Verilator is therefore not used.
- The report is plain Markdown compiled to PDF via pandoc’s default LaTeX template — no custom template, stylesheet or filter is used.